P3020R0
2023-09 Library Evolution Poll Outcomes

Published Proposal, 2023-10-15

Authors:

Inbal Levi - Library Evolution Chair (MPGC Services LTD)

Fabio Fracassi - Library Evolution Assistant Chair (CODE University of Applied Sciences)

Ben Craig - Library Evolution Assistant Chair (Raven)

Billy Baker - Library Evolution Incubator Chair (NVIDIA)

Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair (Argonne National Laboratory)

Corentin Jabot - Library Mailing List Review Manager

Source:

GitHub

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

3.5 Poll 5: Send "[P2819R1] Add tuple protocol to complex" to Library Working Group for C++26.

3.6 Poll 6: Send "[P2821R4] `span.at()`" to Library Working Group for C++26.

3.7 Poll 7: Send "[P2833R1] Freestanding Library: inout expected span" to Library Working Group for C++26.

3.8 Poll 8: Send "[P2836R1] `std::basic_const_iterator` should follow its underlying type's convertibility" to Library Working Group for C++26 and as a DR for C++23.

3.9 Poll 9: Send "[P2868R1] Remove Deprecated `std::allocator` Typedef From C++26" to Library Working Group for C++26.

3.10 Poll 10: Send "[P2870R1] Remove `basic_string::reserve()` From C++26" to Library Working Group for C++26.

3.11 Poll 11: Send "[P2871R2] Remove Deprecated Unicode Conversion Facets From C++26" to Library Working Group for C++2

3.12 Poll 12: Send "[P2905R2] Runtime format strings" to Library Working Group for C++26 and as a DR for C++23.

3.13 Poll 13: Send "[P2918R1] Runtime format strings II" to Library Working Group for C++26.

3.14 Poll 14: Send "[P2937R0] Freestanding: Remove `strtok`" to Library Working Group for C++26.

3.15 Poll 15: Send "[P2909R2] Fix formatting of code units as integers (Dude, where's my char?)" to Library Working Group for C++26 and as a DR for C++23.

References

Informative References

§ 1. Introduction

In 2023-09, the C++ Library Evolution group conducted a series of electronic decision polls [P2972R0]. This paper provides the results of those polls and summarizes the results.

In total, 25 people participated in the polls. Some participants opted to not vote on some polls (Poll 2, Poll 8, Poll 12, Poll 13, had a low participation rate). Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.

- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P0843R9] `inplace_vector`" to Library Working Group for C++26.	12	6	2	1	1	Consensus in favor.
Poll 2: Send "[P1068R9] Vector API for random number generation" to Library Working Group for C++26.	6	7	2	0	0	Strong consensus in favor.
Poll 3: Send "[P2447R4] `std::span` over an initializer list" to Library Working Group for C++26.	7	12	0	1	1	Consensus in favor.
Poll 4: Send "[P2591R4] Concatenation of strings and string views" to Library Working Group for C++26.	8	11	2	1	0	Strong consensus in favor.
Poll 5: Send "[P2819R1] Add tuple protocol to complex" to Library Working Group for C++26.	14	5	1	0	0	Strong consensus in favor.
Poll 6: Send "[P2821R4] `span.at()`" to Library Working Group for C++26.	9	9	2	0	2	Consensus in favor.
Poll 7: Send "[P2833R1] Freestanding Library: inout expected span" to Library Working Group for C++26.	9	8	0	0	1	Consensus in favor.
Poll 8: Send "[P2836R1] `std::basic_const_iterator` should follow its underlying type's convertibility" to Library Working Group for C++26 and as a DR for C++23.	11	4	0	0	0	Strong consensus in favor.
Poll 9: Send "[P2868R1] Remove Deprecated `std::allocator` Typedef From C++26" to Library Working Group for C++26.	15	5	1	0	0	Strong consensus in favor.
Poll 10: Send "[P2870R1] Remove `basic_string::reserve()` From C++26" to Library Working Group for C++26.	16	3	1	1	0	Strong consensus in favor.
Poll 11: Send "[P2871R2] Remove Deprecated Unicode Conversion Facets From C++26" to Library Working Group for C++26.	13	5	1	0	0	Strong consensus in favor.
Poll 12: Send "[P2905R2] Runtime format strings" to Library Working Group for C++26 and as a DR	8	8	0	0	0	Strong consensus in

for C++23.						favor.
Poll 13: Send "[P2918R1] Runtime format strings II" to Library Working Group for C++26.	10	4	0	2	0	Consensus in favor.
Poll 14: Send "[P2937R0] Freestanding: Remove `strtok`" to Library Working Group for C++26.	14	4	1	0	1	Consensus in favor.
Poll 15: Send "[P2909R2] Fix formatting of code units as integers (Dude, where's my char?)" to Library Working Group for C++26 and as a DR for C++23.	10	8	1	0	0	Strong consensus in favor.

All the polls have consensus in favor, and will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P0843R9] `inplace_vector`" to Library Working Group for C++26.

> This is a much needed vocabulary type. Now if only clump/small_vector would get in, then we'd have the complete set.
>
> — Strongly Favor

> This will be a useful addition although I feel the unsafe unchecked_push_back is insufficiently motivated based on benchmark data I've seen.
>
> — Strongly Favor

> This will be a useful addition although I feel the unsafe unchecked_push_back is insufficiently motivated based on benchmark data I've seen.
>
> — Weakly Favor

> This has some performance wins, and has the potential to add a resizable container to freestanding. I’m not thrilled with the unchecked calls from a security perspective, but they are far better than any of the other alternatives that have a chance to make it through the committee.
>
> — Weakly Favor

> Not sure this has had enough time to bake with the current wording. In particular, I believe that the "constexpr" part is currently unimplementable. But, LWG can always send it back if they agree.
>
> — Neutral

> I have some concerns regarding the error handling using exceptions, I think this paper requires more discussion
>
> — Weakly Against

> There is are very few scenarios in which bad_alloc is useful, one of them is by logging that the system has run out of memory. This paper nullifies that use cases by using bad_alloc as the exception it choose to throws, despite having nothing to do at all whatsoever with allocations. What is the point of having a complex hierarchy with bases classes and virtual methods if we are not going to use it correctly? The paper is not motivated, or rather the motivation for this paper ie supporting "environment which can’t allocate" run afoul if its design "everything throws". In its current form, this is unlikely to be adopted by the game industry, the embedded community and other such constrained scenario. This type is therefore not fit for purpose.
>
> — Strongly Against

§ 3.2. Poll 2: Send "[P1068R9] Vector API for random number generation" to Library Working Group for C++26.

> The current one-element-at a-time interface pessimizes for both vectorization and parallelism. The paper’s new interface corrects this, and also updates random number generation for C++ Ranges.
>
> — Strongly Favor

> It allows to make standard RNG faster even without low-level API if the quality of implementation is good. All the comments were address. Let’s ship it!
>
> — Strongly Favor

> It is unclear to me whether we have implementation experience with exactly what’s being proposed.
>
> — Weakly Favor

> I’m not yet convinced this is required by a large group of standard library users
>
> — Weakly Favour

> The wording looks dubious.
>
> — Neutral

§ 3.3. Poll 3: Send "[P2447R4] `std::span` over an initializer list" to Library Working Group for C++26.

> This is how it should be. Simple and expected behavior. While incomplete due to lack of guaranteed storage duration, this remains a step in the right direction.
>
> — Strongly Favor

> A initializer_list is essentially an immutable view over an fixed-size array. It seems like an oversight, that such a type wouldn’t be compatible with span.
>
> — Strongly Favor

> Every time a class is introduced, we should ask if a constructor from initializer_list makes sense or not, because adding them "after the fact" isn’t always possible. As an argument type, span tries to replace a const vector , and span should’ve had initializer_list construction from day one.
>
> — Strongly Favor

> It would be unfortunate to have to recommend the `1,2,3` syntax. The "void two(span);" example is the most concerning incompatibility, but the general rule of initializer lists having runtime size is probably sufficient to avoid most confusion there.
>
> — Weakly Favor

> Breaking code is a little bit scary.
>
> — Weakly Favor

> This should not be a DR. I read the poll as if it isn't.
>
> — Weakly Favor

> The proposal does improve ergonomics. I'm a bit troubled by Section 4.2 ("The initializer_list ctor has high precedence"), in that it might discourage use of spans with compile-time sizes. On the other hand, there is a straightforward work-around. I wonder if we could solve this issue with a deduction guide that uses is_constant_evaluated() to determine whether calling .size() on the initializer_list results in a constant expression.
>
> — Weakly Favor

> Long time in waiting. There is some worry (hence the WF vote) about the cases of broken code, some of them not entirely contrived, but the benefit in both ease of writing and language consistency (and possibly even performance) clearly supersedes that.
>
> — Weakly Favor

> (...) I believe this feature doesn't bring so much value but on the other hand it introduces the breaking change and enables simpler use-cases like `std::span<const int> s{1,2,3};` which is error prone. Yeah, it's better than `std::span<int> s{1,2,3};` because of `const` and far better than `std::span s{1,2,3};` and doesn't work because deduction guides are not enabled for this use-case (fortunately) but still I believe danger for the user is more than the value since the applicability of the API is pretty narrow to me. On the other hand, we already have precedence in the standard where class initialized with temporary could immediately dangle. That's why I am weakly against. If this papers lands, so be it.
>
> — Weakly Against

> span is not a container and should not pretend to be one.
>
> — Strongly Against

§ 3.4. Poll 4: Send "[P2591R4] Concatenation of strings and string views" to Library Working Group for C++26.

> I’ve lost track of how many times I had to workaround these missing operators. While it may have been good intentioned to leave them out until we can come up with a lazy concatenation design, said feature never materialized and most probably shouldn’t be spelled as operator+ to begin with.
>
> — Strongly Favor

> This improves ergonomics and teachability. I am convinced by the arguments in Section 2.1 ("Why are those overloads missing in the first place?") why we should not delay adding this feature in the name of a nonexistent future proposal that could not be implemented fully anyway.
>
> — Strongly Favor

> I strongly believe that we should NOT be updating the __cpp_lib_string_view feature test macro, as we are making NO changes to string_view. We should add a new feature test macro instead (as none of the string ones look to be appropriate). (I would have voted SF instead of WF had it been a different feature test macro). Also, breaking user code is a little bit scary.
>
> — Weakly Favor

> Fixes an inconsistency in the library users have been complaining about (https://stackoverflow.com/q/44636549/471164).
>
> — Weakly Favor

> The proposal is the only reasonable interpretation of the syntax (and its discussion of the history adequately explains the omission as no longer relevant).
>
> — Weakly Favor

> Slightly hesitant due to the conversion-related risks. Overall though this is a highly missed addition, long time due. With this we will no longer have to guide devs to define string constants as constexpr string_view – but only if they don't need them to be concatenated, in which case the unfortunate guidance becomes to use const std::string instead.
>
> — Weakly Favor

> It does fix a mistake, but unimportant.
>
> — Neutral

> Worried about the issues raised in Annex C; I'd prefer to see just the string + string_view overloads and not the additional overloads specified by [tab:string.op.plus.string_view.overloads].
>
> — Weakly Against

§ 3.5. Poll 5: Send "[P2819R1] Add tuple protocol to complex" to Library Working Group for C++26.

> This is how it should be. Simple and expected behavior. It is expected that new features will be consistent with the rest of the language. As such this is more a spec fix or completion of previous work.
>
> — Strongly Favor

> Good facility for std::complex. Assuming the comment on tuple-like concept is addressed I have no objections to that. Hopefully, one day we fix the tuple protocol to make it generic enough but it's a separate story.
>
> — Strongly Favor

> There was a concern about using hidden friends (although I’m not sure it was brought up formally): The main difference between having a normal function vs hidden friend is whether or not it is findable for types that are convertible to complex .
>
> E.g. given: struct my_imaginary { double scale; operator std::complex () const noexcept { return {0, scale}; } };
>
> Then the following will only work if std::complex has non-hidden get() functions: my_imaginary x; auto [r, i] = x; // error: If get() is a hidden friend.
>
> — Strongly Favor

> It makes sense for complex to be decomposable but should it be tuple_like?
>
> — Weakly Favor

> Looks acceptable. I can’t get excited about using structured bindings.
>
> — Weakly Favor

> Applying get<0>(c) to complex values may spoil redability of mathematically intensive code, but at the same the paper adds better integration with other standard facilities. Thus neutral.
>
> — Neutral

§ 3.6. Poll 6: Send "[P2821R4] `span.at()`" to Library Working Group for C++26.

> I’ve already used "span.at()" a few times in code bases, just so that I can be reminded by the compiler that it doesn’t exist yet.
>
> — Strongly Favor

> While it doesn’t bring much value to std::span itself it does bring the value to a generic code and it makes API more consistent. Since we can now add classes partially to freestanding I don’t have any problems with adding this method.
>
> — Strongly Favor

> We can entertain lengthy discussions about how and why std::span differs from the original gls::span, but at this point we should simply strive for consistency.
>
> — Strongly Favor

> It is embarrassing not to have it. It is safe and consistent. We need more safe functions instead of unsafe ones.
>
> — Strongly Favor

> This addition makes span more consistent with vector. I also appreciate that the author has taken the time to address freestanding. However, I disagree that "a function that throws when the input is out of bounds" is necessarily "safer" than a function with a nontrivial precondition. I’m not an expert in functional safety, but I’ve heard such experts share my opinion. My concern only addresses this one motivation for the paper; making span consistent with vector justifies the paper in itself.
>
> — Weakly Favor

> So long as we’re happy having classes have incomplete interfaces in freestanding, this is only reasonable based on consistency, since there’s no cost to having the feature.
>
> — Weakly Favor

> Not convinced that this is worth adding.
>
> — Neutral

> This is extremely poorly motivated in my view. Throwing exception of developer error is not something we should encourage, and it certainly does not encourage safety. Consistency with past mistakes is also not a very convincing argument. Ask your local standard library implementer an harden build that terminates on out-of-bound. These things exist and they are great!
>
> — Strongly Against

> This is safety theatre (a term analogous to the well understood term "security theater"). MISRA doesn’t require usage of .at() and I don’t see C++ engineers adopting .at() any time soon. If we want to reduce C++ vulnerabilities due to memory safety violations, we should attack that problem directly instead of making changes that make us feel good, but do nothing to solve our problems.
>
> — Strongly Against

3.7. Poll 7: Send "[P2833R1] Freestanding Library: inout expected span" to Library Working Group for C++26.

> freestanding has needed some kind of error handling mechanism for awhile, so this will be a big win.
>
> — Strongly Favor

> This will aid span’s adoption which is essential since it is a fundamental type.
>
> — Strongly Favor

> Would have been nice to have the freestanding features done in fewer papers to save time for LEWG.
>
> — Weakly Favor

> We don’t use freestanding, but these additions seem reasonable and useful for those that do.
>
> — Weakly Favor

> The addition of these features to the freestanding subset should be non-controversial.
>
> — Weakly Favor

> I’m unconvinced that spending time on the freestanding dialect of C++ is in the interest of our users.
>
> — Strongly Against

3.8. Poll 8: Send "[P2836R1] `std::basic_const_iterator` should follow its underlying type's convertibility" to Library Working Group for C++26 and as a DR for C++23.

> This fixes a deficiency in the current C++ Standard. I agree that this change should be considered library DR.
>
> — Strongly Favor

> We need to add uniformity, and "wrapper types" should behave as the wrapped type
>
> — Strongly Favor

> I like not breaking C++20 code
>
> — Weakly Favor

> I'm not a fan of the DR part.
>
> — Weakly Favor

> The proposed conversions could in theory produce undesired complications, but I haven't checked.
>
> — I do not want to participate in this poll

3.9. Poll 9: Send "[P2868R1] Remove Deprecated `std::allocator` Typedef From C++26" to Library Working Group for C++26.

> Once a good idea, but now std::allocator is no longer necessary.
>
> — Strongly Favor

> Although implementations will likely leave these in, the purpose of deprecation is ultimately to remove things, and so this should be done.
>
> — Strongly Favor

> We must remove or control baggage if C++ is to grow, change and remain relevant.
>
> — Strongly Favor

> I'll finger the elephant in the room: DRY violation. This is an attribute, so its single source of truth should be allocator_traits, it has no business getting itself duplicated anywhere else.
>
> — Weakly Favor

> We have experience with the removed members in std::allocator.
>
> — Weakly Favor

3.10. Poll 10: Send "[P2870R1] Remove `basic_string::reserve()` From C++26" to Library Working Group for C++26.

> I'm not sure why this function even exists in the first place. The default parameter that was originally supplied looks like a design bug - especially as it was exclusive to basic_string. Given that we have a replacement since C++11 and it isn't even functional anymore since C++20 I see no reason to retain this overload.
>
> — Strongly Favor

> Although implementations will likely leave these in, the purpose of deprecation is ultimately to remove things, and so this should be done.
>
> — Strongly Favor

> We must remove or control baggage if C++ is to grow, change and remain relevant.
>
> — Strongly Favor

> The real semantic change (which is justifiable) has already occurred. This is, if anything, helpfully alerting people to the fact.
>
> — Weakly Favor

> I’m worried about the reported lack of deprecation warnings, and about the paper missing any research into the amount of existing code which may be broken by this. I think straight up removal may surprise some users by providing them no immediate clue as to how to modify their code. Therefore the path forward should be one cycle in which this function is implemented via a static_assert(false) with a relevant error message, followed by complete removal only in the next cycle (C++29).
>
> — Weakly Against

3.11. Poll 11: Send "[P2871R2] Remove Deprecated Unicode Conversion Facets From C++26" to Library Working Group for C++2

> Having these seems worse than having nothing.
>
> — Strongly Favor

> The implementation is broken in MSVC at one point anyway.
>
> — Strongly Favor

> Regularly removing deprecated functionality is a good practice.
>
> — Strongly Favor

> This feature has never fundamentally worked correctly. Deprecation and, now, removal even without replacement is appropriate.
>
> — Weakly Favor

> Due to lack of usage experience I can’t comment on these facets.
>
> — Neutral

§ 3.12. Poll 12: Send "[P2905R2] Runtime format strings" to Library Working Group for C++26 and as a DR for C++23.

> This is the right thing to do. We should also make this a DR for C++23. I can't think of any correct code that would be broken by doing so.
>
> — Strongly Favor

> Improves safety of a formatting API by preventing some lifetime issues.
>
> — Strongly Favor

> I'm not a fan of the DR part, but it seems more plausible here than in some other cases.
>
> — Weakly Favor

> Preventing dangling is great but it is concerning that we continue to retroactively modify std::format two standard cycles after the feature was initially added.
>
> — Weakly Favor

> Obviously being able to use such strings is valuable, but the change could easily be unfortunately invasive; I haven't tried to vet it independently.
>
> — I do not want to participate in this poll

§ 3.13. Poll 13: Send "[P2918R1] Runtime format strings II" to Library Working Group for C++26.

> This is a good idea. Separating this from P2905 (the safety fix alone) was also a good idea, so that P2905 could be made a DR, while P2918 can be introduced "normally" in C++26.
>
> — Strongly Favor

> A straight-forward extension to std::format that doesn't require changing existing standards and enables to-date missing functionality.
>
> — Strongly Favor

> A very important addition to std::format that allows to explicitly opt out of compile-time checks instead of misusing type erased APIs.
>
> — Strongly Favor

> Has implementation and usage experience.
>
> — Strongly Favor

> The new API seems a bit confusing, but not enough to preclude the important functionality.
>
> — Weakly Favor

> Users expect to be able to use a std::string as the first argument of std::format. If we lack the language facilities to provide this, we should add them instead of spending time with unsatisfying workarounds.
>
> — Weakly Against

> Why not std::runtime_format(str, 42)
>
> — Weakly Against

§ 3.14. Poll 14: Send "[P2937R0] Freestanding: Remove `strtok`" to Library Working Group for C++26.

> One foot-gun less ... strtok should actually be removed from the entire standard library.
>
> — Strongly Favor

> This function should not only be removed from freestanding, it should be yanked from the standard library as a whole.
>
> — Strongly Favor

> Keep C++ freestanding in line with C freestanding.
>
> — Strongly Favor

> One could argue that strtok on freestanding should just not be thread-safe at all, but the fact that C doesn't expect it to be there tips the balance in favor of removing it. (Who uses it anyway?)
>
> — Weakly Favor

> Would have been nice to have the freestanding features done in fewer papers to save time for LEWG
>
> — Weakly Favor

> I'm unconvinced that spending time on the freestanding dialect of C++ is in the interest of our users.
>
> — Strongly Against

§ 3.15. Poll 15: Send "[P2909R2] Fix formatting of code units as integers (Dude, where's my char?)" to Library Working Group for C++26 and as a DR for C++23.

> This feels like the right thing to do, even if it is a breaking change. It's important for Standard Library output to be consistent.
>
> — Strongly Favor

> Spec fixes should be done, especially when there is negligible breakage.
>
> — Strongly Favor

> Fixes a problematic behavior in formatting characters as integers and makes formatting consistent among platforms.
>
> — Strongly Favor

> Improved consistency is great but we are yet again retroactively changing the meaning of std::format.
>
> — Weakly Favor

> No one expects code units to actually be negative and relies on two's complement (and little real arithmetic) to make that never matter in practice.
>
> — Weakly Favor

> Runtime change in behavior of C++20 facilities makes me really nervous, even when the change in an improvement.
>
> — Neutral

§ References

§ Informative References

[P0843R9]
Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. *inplace_vector*. 14 September 2023. URL: https://wg21.link/p0843r9

[P1068R9]
Ilya Burylov, Pavel Dyakov, Ruslan Arutyunyan, Andrey Nikolaev, Alina Elizarova. *Vector API for random number generation*. 14 September 2023. URL: https://wg21.link/p1068r9

[P2447R4]
Arthur O'Dwyer, Federico Kircheis. *std::span over an initializer list*. 14 May 2023. URL: https://wg21.link/p2447r4

[P2591R4]
Giuseppe D'Angelo. *Concatenation of strings and string views*. 11 July 2023. URL: https://wg21.link/p2591r4

[P2819R1]
Michael Florian Hava, Christoph Hofer. *Add tuple protocol to complex*. 14 July 2023. URL: https://wg21.link/p2819r1

[P2821R4]
Jarrad J. Waterloo. *span.at()*. 26 July 2023. URL: https://wg21.link/p2821r4

[P2833R1]

Ben Craig. *Freestanding Library: inout expected span*. 19 August 2023. URL: https://wg21.link/p2833r1

[P2836R1]

Christopher Di Bella. *std::basic_const_iterator should follow its underlying type's convertibility*. 11 July 2023. URL: https://wg21.link/p2836r1

[P2868R1]

Alisdair Meredith. *Remove Deprecated `std::allocator` Typedef From C++26*. 15 August 2023. URL: https://wg21.link/p2868r1

[P2870R1]

Alisdair Meredith. *Remove `basic_string::reserve()` From C++26*. 16 August 2023. URL: https://wg21.link/p2870r1

[P2871R2]

Alisdair Meredith. *Remove Deprecated Unicode Conversion Facets From C++26*. 15 September 2023. URL: https://wg21.link/p2871r2

[P2905R2]

Victor Zverovich. *Runtime format strings*. 23 July 2023. URL: https://wg21.link/p2905r2

[P2909R2]

Victor Zverovich. *Fix formatting of code units as integers (Dude, where's my char?)*. 16 September 2023. URL: https://wg21.link/p2909r2

[P2918R1]

Victor Zverovich. *Runtime format strings II*. 15 July 2023. URL: https://wg21.link/p2918r1

[P2937R0]

Ben Craig. *Freestanding: Remove strtok*. 2 July 2023. URL: https://wg21.link/p2937r0

[P2972R0]

Inbal Levi, Ben Craig, Fabio Fracassi, Corentin Jabot, Nevin Liber, Billy Baker. *2023-09 Library Evolution Polls*. 18 September 2023. URL: https://wg21.link/p2972r0